Day 1

Learning Python

Basic to Advance

Chapter 1: Introduction

Topics Covered:

- What is Python?
- Features of Python
- Applications of Python
- Python Installation
- First Python Program
- Quick Recap

python™

NEXT

What is Python?

Python is a powerful, high-level programming language designed to be easy to read and simple to implement.

It was created by Guido van Rossum and first released in 1991.

Python emphasizes code readability with its use of significant indentation.

Guido Van Rossum

Creator of Python!

Features of Python

- **Easy to Read and Write:** Python's syntax is straightforward, resembling plain English, making it beginner-friendly.
- **Interpreted Language:** Python code is executed line by line, which helps in debugging and rapid development.
- **Versatile:** Python is used in various fields like web development, data analysis, AI, automation, and more.
- **Rich Ecosystem:** It has a large standard library and numerous third-party packages available.

Applications of Python

- **Web Development:** Build dynamic websites and applications using frameworks like Django and Flask.
- **Data Science:** Analyze and visualize data using libraries like Pandas and Matplotlib.
- **Machine Learning**: Create intelligent systems and algorithms with tools like Scikit-learn and TensorFlow.
- **Automation:** Automate repetitive tasks and processes with simple scripts.

Python Installation

- **Download Python:** Go to python.org and download the latest version for your operating system.

- **Run the Installer:** Open the downloaded file and follow the installation instructions. Make sure to check the box that says "Add Python to PATH."

- **Verify Installation:** Open your terminal (Command Prompt on Windows or Terminal on macOS/Linux) and type python --version. You should see the installed Python version number.

First Python Program

Let's create and run your first Python program using Visual Studio Code (VS Code):

- Go to the folder where you want to save your program.
- Right-click in the folder and select “Open with Code” from the context menu.
- VS Code will open with the selected folder. Inside VS Code, create a new file named first_program.py.
- Write the Code:

```
print("Hello, world!")
```

- Press Ctrl + S (Windows/Linux) or Cmd + S (macOS) to save the file.
- Click on the "Run" button in VS Code, or use the shortcut Ctrl + Alt + N.
- Look at the terminal output in VS Code; you should see “Hello, World!” printed on the screen.

Let’s break down the code:

- print: This is a built-in Python function that outputs data to the screen. In this case, it's used to display text.
- "Hello, world!": This is a string, a type of data in Python that represents text. The text inside the quotes is what will be displayed when the program runs.

Putting it all together, print("Hello, world!") tells Python to display the text "Hello, world!" on the screen.

Day 2 Learning Python

Basic to Advance

Chapter 2: Modules, Comments, & Pip

Topics Covered:

- Modules in Python
- Comments in Python
- What is Pip?
- Using Pip to install packages
- Example of a module

Modules in Python

A module is simply a file containing Python code that you can reuse in other programs.

Modules help keep your code organized and manageable.

You can create your own modules or use built-in ones like math or os.

Example: If you save some functions in a file named mymodule.py, you can import them into another script with import mymodule.

Comments in Python

Comments are notes you can add to your code to make it more understandable.

Python ignores comments when running the program, so they don't affect how your code works.

Single-line Comment: Start the line with #.

```
# This is a comment
print("Hello, World!")
```

Multi-line Comment: Use triple quotes (''' or """) for longer comments.

```
'''
This is a multi-line comment
that spans several lines.
'''
print("Hello, again!")
```

What is Pip?

Pip is a package manager for Python.

It allows you to install and manage additional libraries and packages that aren't included with Python by default.

With Pip, you can easily add tools and libraries to your project.

- Think of Pip as an app store for Python packages.

Using Pip to Install Packages

You can install packages using Pip directly from your terminal or command prompt. Here's how:

- **Open Terminal:** (Command Prompt on Windows).
- **Type the Install Command:** For example, to install the popular requests package for making HTTP requests, type:

```
pip install requests
```

- **Verify Installation:** Once installed, you can import and use the package in your Python scripts.

Example of a Module

Let's say you have a file named mymodule.py with the following content:

```
# mymodule.py
def greet(name):
    return f"Hello, {name}!"
```

You can use this module in another script:

```
import mymodule

print(mymodule.greet("Alice"))
```

This will output Hello, Alice!.

Day 3 Learning Python

Basic to Advance

Chapter 3: Variables, Data Types & Keywords

Topics Covered:

- What is a Variable?
- How to create a Variable
- Identifiers in Python
- Data Types in Python
- Keywords in Python

What is a Variable?

A variable in Python is like a container that holds data.

When you create a variable, you're giving a name to a value that you want to store and use later in your program.

Think of it like labeling a box to remember what's inside.

How to Create a Variable?

You can create a variable by simply assigning a value to it using the = sign.

```
age = 25
name = "Alice"
is_student = True
```

In this example:

- age is a variable storing the value 25 (an integer).
- name is a variable storing the value "Alice" (a string, which is text).
- is_student is a variable storing the value True (a boolean, which represents true/false).

Key Points:

- Variables can store different types of data like numbers, text, or even more complex data.
- Once a variable is created, you can use it throughout your program.

What is an Identifier?

Identifiers are the names you give to variables, functions, classes, and other objects in Python.

They are like labels that you use to refer to these objects.

Rules for Naming Identifiers:

- Identifiers can only contain letters (a-z, A-Z), digits (0-9), and underscores (_).
- An identifier must start with a letter or an underscore, but not with a digit.
- Identifiers are case-sensitive, meaning myVar and myvar are considered different.
- Python has some reserved words (called keywords) that you cannot use as identifiers.

Examples of Valid Identifiers:

```
my_variable = 10
totalAmount = 100
_isValid = True
```

Examples of Invalid Identifiers:

```
2ndValue = 50
# Invalid because it starts with a digit
total-amount = 200
# Invalid because it contains a hyphen
```

What are Data Types?

Data types define the kind of data a variable can hold. Python has several built-in data types that help you work with different kinds of information.

Common Data Types in Python:

- Integers (int): Whole numbers without a decimal point.
  - **Example:** 5, -10, 1000
  - **Usage:** age = 25
- Floating-point numbers (float): Numbers with a decimal point.
  - **Example:** 3.14, -0.5, 10.0
  - **Usage:** price = 19.99
- Strings (str): Text enclosed in single or double quotes.
  - **Example:** "hello", 'Python', "123"
  - **Usage:** name = "Alice"

- **Booleans (bool):** Represents True or False.
  - Example: True, False
  - Usage: is_logged_in = False
- **Lists (list):** Ordered collections of items, which can be of mixed data types.
  - Example: [1, 2, 3], ["apple", "banana", "cherry"]
  - Usage: fruits = ["apple", "banana", "cherry"]
- **Tuples (tuple):** Ordered collections of items, similar to lists, but immutable (cannot be changed).
  - Example: (1, 2, 3), ("apple", "banana", "cherry")
  - Usage: coordinates = (10, 20)
- **Dictionaries (dict):** Collections of key-value pairs, where each key is unique.
  - Example: {"name": "Alice", "age": 25}
  - Usage: person = {"name": "Alice", "age": 25}

Examples:

```python
# Integer
age = 30

# Float
price = 19.99

# String
name = "John"

# Boolean
is_student = True

# List
numbers = [1, 2, 3, 4, 5]

# Tuple
dimensions = (1920, 1080)

# Dictionary
person = {"name": "Alice", "age": 25}
```

What are Keywords?

Keywords are reserved words in Python that have a special meaning and purpose.

You cannot use them as identifiers (variable names, function names, etc.).

These keywords are part of the language syntax and structure.

Examples of Python Keywords:

- if, else, elif: Used for conditional statements.
- for, while: Used for loops.
- def: Used to define a function.
- return: Used to return a value from a function.
- True, False: Boolean values.
- import: Used to import modules.

Complete List of Python Keywords:

Keyword	Keyword	Keyword	Keyword
False	await	else	import
None	break	except	in
True	class	finally	is
and	continue	for	lambda
as	def	from	nonlocal
assert	del	global	not
async	elif	if	or
pass	raise	return	try
while	with	yield	

Day 4 Learning Python

Basic to Advance

Chapter 3: Operators in Python

Topics Covered:

- Arithmetic Operators
- Comparison Operators
- Assignment Operators
- Logical Operators
- Bitwise Operators
- Membership Operators
- Identity Operators

Operators in Python

Operators in Python are symbols or special characters used to perform specific operations on variables and values.

Python provides various types of operators to manipulate and work with different data types.

Here are some important categories of operators in Python:

- Arithmetic Operators
- Comparison Operators
- Assignment Operators
- Logical Operators
- Bitwise Operators
- Membership Operators
- Identity Operator

Arithmetic Operators

Arithmetic operators in Python are used to perform mathematical calculations on numeric values. The basic arithmetic operators include:

- **Addition (+):** Adds two operands together.
  Example: a + b adds a and b.

- **Subtraction (-):** Subtracts the second operand from the first.
  Example: a - b subtracts b from a.

- **Multiplication (*):** Multiplies two operands.
  Example: a * b multiplies a and b.

- **Division (/):** Divides the first operand by the second.
  Example: a / b divides a by b.

- **Modulus (%):** Returns the remainder of the division.
  Example: a % b gives the remainder of a divided by b.

- Exponentiation (): ** Raises the first operand to the power of the second.
  Example: a ** b raises a to the power of b.

- Floor Division (//): Divides the first operand by the second and rounds down to the nearest integer.
  Example: a // b divides a by b and returns the floor value.

Comparison Operators

Comparison operators are used to compare two values. The result is either True or False (a Boolean value).

- Equal (==): Checks if two operands are equal.
  Example: a == b is True if a is equal to b.

- Not Equal (!=): Checks if two operands are not equal.
  Example: a != b is True if a is not equal to b.

- **Greater Than (>):** Checks if the first operand is greater than the second.
  Example: a > b is True if a is greater than b.

- **Less Than (<):** Checks if the first operand is less than the second.
  Example: a < b is True if a is less than b.

- **Greater Than or Equal To (>=):** Checks if the first operand is greater than or equal to the second.
  Example: a >= b is True if a is greater than or equal to b.

- **Less Than or Equal To (<=):** Checks if the first operand is less than or equal to the second.
  Example: a <= b is True if a is less than or equal to b.

Assignment Operators

Assignment operators are used to assign values to variables.

- **Equal (=):** Assigns the value of the right operand to the left operand.
  Example: a = 10 assigns 10 to a.

- **Add and Assign (+=):** Adds the right operand to the left operand and assigns the result to the left operand.
  Example: a += 5 is the same as a = a + 5.

- **Subtract and Assign (-=):** Subtracts the right operand from the left operand and assigns the result to the left operand.
  Example: a -= 5 is the same as a = a - 5.

- **Multiply and Assign (*=):** Multiplies the right operand by the left operand and assigns the result to the left operand.
  Example: a *= 5 is the same as a = a * 5.

- **Divide and Assign (/=):** Divides the left operand by the right operand and assigns the result to the left operand.
  Example: a /= 5 is the same as a = a / 5.

- **Modulus and Assign (%=):** Takes the modulus of the left operand with the right operand and assigns the result to the left operand.
  Example: a %= 5 is the same as a = a % 5.

- **Exponentiate and Assign (=):**** Raises the left operand to the power of the right operand and assigns the result to the left operand.
  Example: a **= 5 is the same as a = a ** 5.

- **Floor Divide and Assign (//=):** Performs floor division on the left operand with the right operand and assigns the result to the left operand.
  Example: a //= 5 is the same as a = a // 5.

Logical Operators

Logical operators are used to combine conditional statements. They return True or False.

- **and:** Returns True if both statements are true. Example: a > 5 and b < 10 is True if both conditions are true.
- **or:** Returns True if at least one of the statements is true.
  Example: a > 5 or b < 10 is True if either condition is true.
- **not:** Reverses the result, returns False if the result is True.
  Example: not(a > 5) is True if a is not greater than 5.

Bitwise Operators

Bitwise operators work on bits and perform bit-by-bit operations.

- **AND (&):** Sets each bit to 1 if both bits are 1. Example: a & b.
- **OR (|):** Sets each bit to 1 if one of the bits is 1. Example: a | b.
- **XOR (^):** Sets each bit to 1 if only one of the bits is 1. Example: a ^ b.
- **NOT (~):** Inverts all the bits. Example: ~a.
- **Left Shift (<<):** Shifts bits to the left by the specified number of positions. Example: a << 2 shifts bits in a two positions to the left.
- **Right Shift (>>):** Shifts bits to the right by the specified number of positions. Example: a >> 2 shifts bits in a two positions to the right.

Membership Operators

Membership operators test if a sequence is present in an object.

- in: Returns True if a sequence with the specified value is present in the object. Example: 'a' in 'apple' is True.

- not in: Returns True if a sequence with the specified value is not present in the object. Example: 'b' not in 'apple' is True.

Identity Operators

Identity operators are used to compare the memory locations of two objects.

- **is:** Returns True if both variables are the same object (i.e., they refer to the same memory location). Example: a is b is True if a and b refer to the same object.

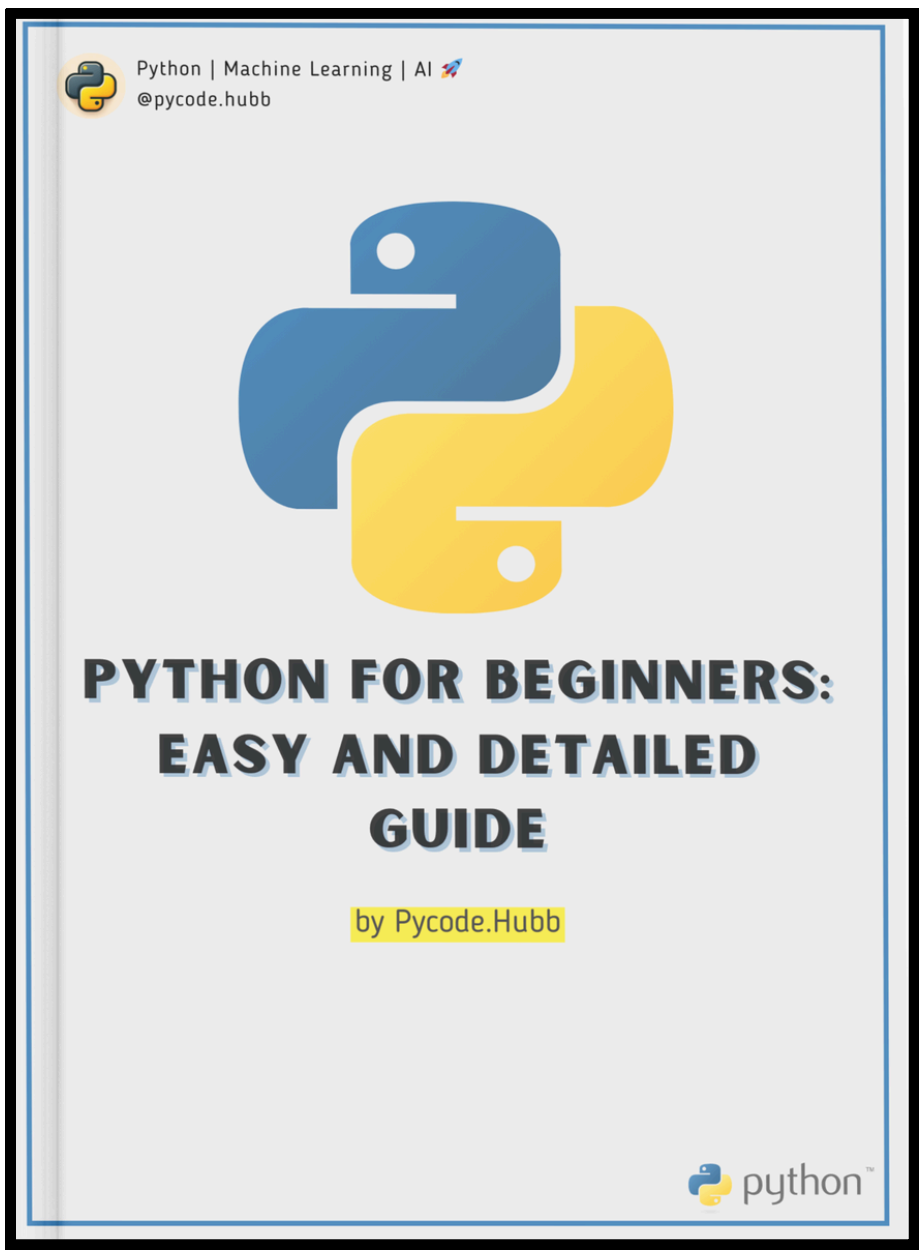
- **is not:** Returns True if both variables are not the same object. Example: a is not b is True if a and b do not refer to the same object.

Python | Machine Learning | AI 🚀
@pycode.hubb

DM For Project Help

UNLOCK THE POWER OF PYTHON WITH OUR ULTIMATE BUNDLE!

Learn Python step-by-step and build real-world projects with our comprehensive guide and 99 project pack.



🔗 Link in the Bio 🔗

python™

NEXT

THANKS FOR READING :)

If you found this post useful, then giving a like and following this page would mean the world to me <3

PYCODE.HUBB

LIKE | COMMENT | SHARE | FOLLOW